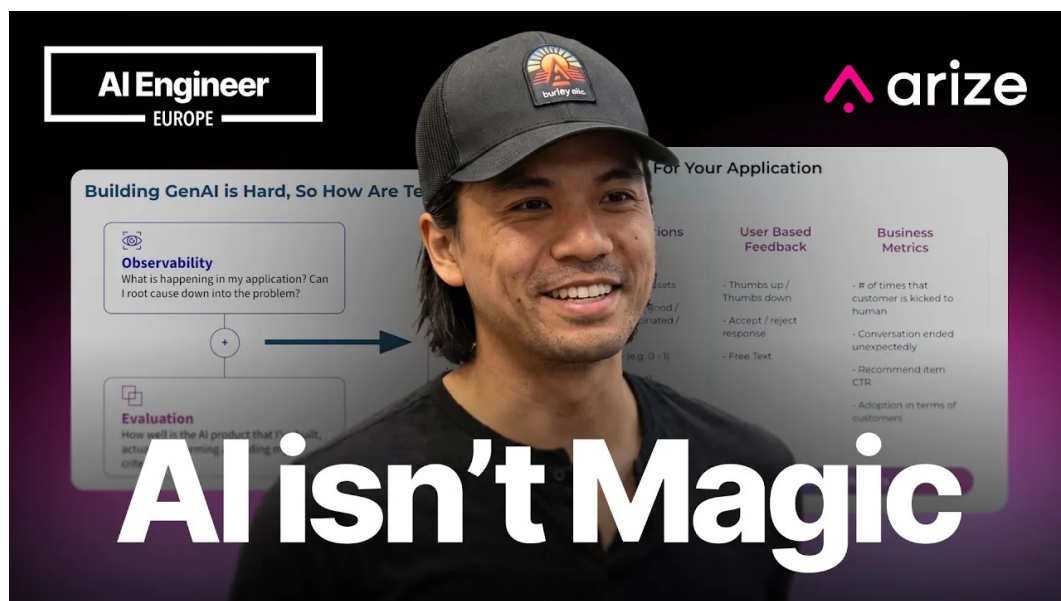


AI 没有魔法：LLM 可观测性、评估与实验的工程闭环

五道口纳什 & Codex

2026 年 6 月 29 日



视频作者/频道：AI Engineer

发布日期：2026 年 6 月 7 日

视频时长：16:32

视频链接：<https://www.youtube.com/watch?v=JsCCrBF7F1g>

主题：LLM observability、evaluation、experimentation platform；演讲者 Dat Ngo, Arize。

PDF 制作 Skill：<https://github.com/wdkns/wdkns-skills>

目录

学习路线	2
1 AI 没有魔法：三件工程事	2
1.1 从企业痛点看三根支柱	3
1.2 为什么这三件事必须连在一起	4
1.3 从本章进入下一章	4
1.4 本章小结	4
2 可观测性：用遥测看见代理系统做了什么	5
2.1 Otel first：先把行为标准化记录下来	5
2.2 Trace、span 与 session：从调用片段到状态过程	6
2.3 Branch、path 与 distributional view：观察许多次运行	7
2.4 Root cause：从路径异常追到因果结构	8
2.5 从本章进入下一章	8
2.6 本章小结	9
3 评价信号：从 LLM-as-a-Judge 到业务指标	9
3.1 LLM as a Judge：灵活但需要校准	9
3.2 确定性评估：便宜、清晰、可自动化	10
3.3 人、用户与业务：质量定义不只属于工程团队	10
3.4 用复合分数作为教学脚手架	11
3.5 从本章进入下一章	11
3.6 本章小结	12
4 评价作用域：Span、Multi-span、Trajectory、Session	12
4.1 Span eval：看一个组件的输入与输出	12
4.2 Multi-span eval：跨组件检查数据传递	13
4.3 Trajectory eval：检查步骤顺序和业务过程	13
4.4 Session level eval：评估状态机与用户体验	14
4.5 Minimal set of evals：评估越多，成本也越高	14
4.6 从本章进入下一章	15
4.7 本章小结	15

5 实验与改进：把观察到的问题变成可验证的变化	15
5.1 从 traces 到 dataset：把失败样本收集起来	16
5.2 四类常见改动	17
5.3 实验比较：关注指标变化与隐藏回归	17
5.4 UI 与程序化实验	17
5.5 从本章进入下一章	18
5.6 本章小结	18
6 自动化飞轮：让 AI 参与观测、评价与改进	18
6.1 CLI、tools、skills：把平台能力交给 coding agent	18
6.2 Alex：在 trace 与 eval 上工作的助手	19
6.3 动态 eval：让系统根据变化提出新的检查	19
6.4 Phoenix 与 Arize AX：两个产品定位	20
6.5 从本章进入下一章	21
6.6 本章小结	21
7 总结与延伸：从演示系统到生产治理	21
7.1 四层治理模型	21
7.2 给不同角色的实践路线	22
7.3 核心风险清单	22
7.4 一页落地清单	22
7.5 最终综合	23
7.6 本章小结	23

学习路线

这份笔记采用金字塔结构组织：先给出结论，再解释机制，随后用演讲中的画面与字幕证据支撑，最后给出团队落地时容易忽略的风险。全篇主线是一个闭环：生产 LLM/agent 系统需要先通过 telemetry 看见事实，再通过 evals 提炼信号，随后通过 experiments 比较改动，最终把有效改动纳入系统。

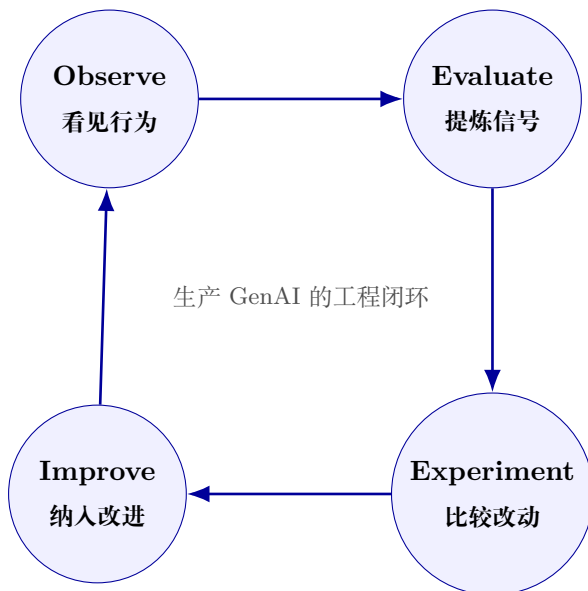


图 1: Observe-Evaluate-Experiment-Improve 飞轮：可观测性提供事实，评估提炼信号，实验比较改动，改进再回到新的观测。

读者如何使用这份笔记

AI 工程师可以重点阅读第 2、4、5、6 章，理解 trace/span/session、eval scope、dataset 和 CLI/agent automation。产品经理和领域专家可以重点阅读第 3、4、7 章，理解质量信号如何来自用户反馈、业务指标和黄金数据集。技术负责人应关注第 5、6、7 章的回归、权限、自动化和治理边界。

1 AI 没有魔法：三件工程事

Dat Ngo 在开场给出的核心判断很直接：生成式 AI 产品想进入生产环境，主要难点可以被拆成三件工程事：看见系统发生了什么，判断这些行为是否可接受，然后用实验验证改动是否真正带来改进。演讲中的记忆钩子是 *AI isn't magic*。这句话在本讲里并非一句口号，它提醒团队把神秘感还原为可检查、可度量、可迭代的工程闭环。

Dat 介绍自己是 Arize AI 的 AI architect，长期和大型企业讨论可观测性、评估与实验。他提到自己在相关工作中花费了大量 token，并从企业客户的转型过程中看到各团队正在构建什么、怎样构建、遇到哪些痛点以及如何修复。这个视角很重要：本讲的内容来自 Arize 的产品演示和企业实践观察，后文所有产品能力都应理解为 Dat/Arize 演示中的主张。

Building GenAI is Hard, So How Are Teams Tackling This?

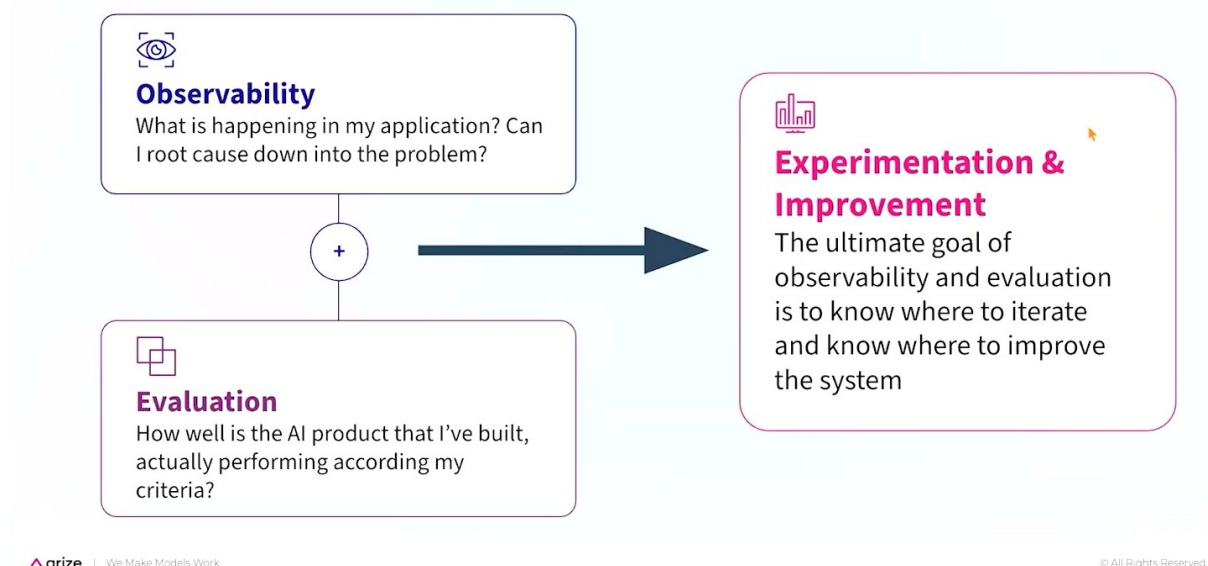


图 2: Dat 用三支柱图把生产 GenAI 的工作拆成 observability、evaluation、experimentation and improvement。¹

Dat 将 GenAI 应用称为 *software reimagined*，也就是“软件被重新想象”。直觉上，LLM、智能体和工具调用似乎带来了全新的不确定性；工程上，团队仍然要处理老问题：系统在运行时做了什么，结果是否符合预期，改动后有没有引入隐藏故障。区别在于对象从确定性的代码路径扩展为提示词、模型、工具、编排逻辑、状态和多轮会话。

本讲的主线

生产 GenAI 的基本闭环可以概括为：

Observe → Evaluate → Experiment → Improve

其中，*Observe* 指通过遥测看见系统行为；*Evaluate* 指把行为转成质量、安全、成本或业务信号；*Experiment* 指对提示词、模型、编排和配置做受控比较；*Improve* 指基于证据迭代系统。

1.1 从企业痛点看三根支柱

第一根支柱是可观性 (*observability*)。它回答的问题是：团队构建的 harness、agent 或应用编排框架内部到底发生了什么？这里的 harness 可以理解为“把模型、提示词、工具、检索、记忆和业务规则装配起来的执行框架”。一个单次 LLM 调用像一次函数调用；一个智能体系统更像一条会分岔的流水线，某一步可能调用检索，某一步可能调用工具，某一步可能把状态交给另一个组件。

第二根支柱是评估 (*evaluation / evals*)。可观性让团队拿到行为记录，evals 则把这些记录转换成信号 (*signal*)。信号可以来自 LLM 裁判、代码检查、人工标注、用户反馈和业务指标。换句话说，trace 本身只是证据，eval 才把证据变成“好、坏、可疑、昂贵、延迟高、业务有效”这些可行动判断。

¹ 视频画面时间区间：00:01:31-00:03:04。若为裁剪图，时间区间仍对应原视频画面。

第三根支柱是实验与改进 (*experimentation & improvement*)。Dat 特别提醒：在非确定性系统里，一个看似修复问题的改动，可能同时造成两个或三个团队没有预料到的回归问题 (*regressions*)。例如，提示词改动让答案更完整，却让延迟变高、成本上升，甚至让某些边界输入的安全性下降。实验的作用就是把“感觉变好了”转成可比较的证据。

隐藏回归的风险

LLM 产品最危险的假象之一是局部样例变好后，团队就认为系统整体改善。真实生产系统有多个目标：质量、延迟、成本、安全、可解释性、用户满意度和业务结果。某个指标上升时，其他指标可能正在恶化。

1.2 为什么这三件事必须连在一起

只做观测，团队会得到大量日志、trace 和 dashboard，却缺少优先级；只做评估，团队可能知道分数变化，却说不清根因；只做实验，团队会不断试改提示词和模型，却没有足够证据解释输赢。三者连起来后，生产 workflow 才完整：先定位行为，再抽取信号，再比较候选改动。

一个直观类比是医院诊疗流程。可观测性像体检和影像，它告诉医生身体内部发生了什么；评估像诊断标准，它把体检结果转成风险判断；实验与改进像治疗方案对照，它验证药物和剂量是否有效。AI 系统同样需要这种“先看见、再判断、再改变”的顺序。

关键术语：智能体与 harness

智能体 (*agent*) 通常具备多步决策、工具调用、状态传递和路径选择能力。Harness 或应用执行框架负责把模型、提示词、工具、检索、记忆与业务逻辑连接起来。生产化智能体 (*productionized agent*) 已经进入真实用户或业务流程，因此需要监控、治理和事故响应。

1.3 从本章进入下一章

Dat 的三支柱框架把问题范围圈定了：AI 产品的生产化不能依赖演示时的一次成功回答，可靠路径是持续的工程闭环。下一章先进入第一支柱：可观测性。只有先通过 OpenTelemetry、traces、spans、sessions 和分布式视角看见智能体系统的真实行为，团队才有资格谈评估和实验。

1.4 本章小结

本章的结论是：*AI isn't magic* 在这里意味着“把魔法感还原成工程闭环”。Dat 从 Arize 面向大型企业的观察出发，将生产 GenAI 的核心工作压缩为三件事：*observability* 看见行为，*evals* 提炼信号，*experimentation & improvement* 验证改动。这个框架也给后文定下边界：所有工具、dashboard、LLM judge、dataset 和 Alex 自动化，都服务于同一个目标，即让团队在非确定性系统中做可证据化的改进。

2 可观测性：用遥测看见代理系统做了什么

可观测性的核心任务是把智能体系统的内部行为变成可检查的遥测证据。Dat 在 W03-W05 中反复强调，代码本身不会自然审计一个 agent 或 harness；真正能留下审计记录（*audit record*）的是 telemetry，尤其是 OpenTelemetry 语境下的 *traces and spans*。

可观测性的工程定义

在本讲语境中，可观测性（*observability*）指团队通过 telemetry、traces、spans、sessions、agent graph、dashboard 和 analytics 观察 AI 系统的行为、路径、状态、错误、延迟、成本与根因的能力。

2.1 Otel first：先把行为标准化记录下来

Dat 称 Arize 的方案是 *Otel first*，即优先采用 OpenTelemetry / Otel 开放遥测模式。OpenTelemetry 可以理解为工程系统中的“统一病历格式”：无论底层是哪个框架、SDK、模型或工具链，只要记录遵循相对标准的 trace/span 结构，后续分析、展示、迁移和集成都更容易。

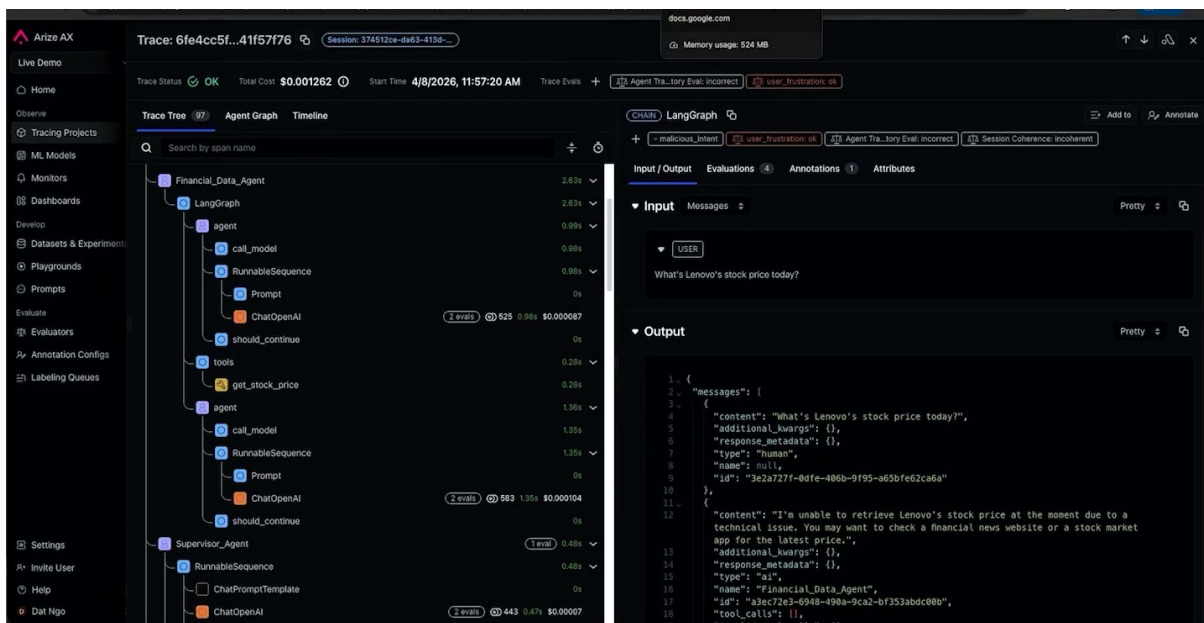


图 3: Arize AX 中的 trace tree 展示了 agent 运行的调用片段、成本、耗时和输入输出面板。²

Dat 提到 *auto instrumenter*，也就是自动埋点器。它的目标是让开发者用很少的代码改动，例如一行集成代码，就能捕获框架或 SDK 周围发生的事情，并生成 OpenTelemetry traces 和 spans。这里的“一行代码”应按演示主张理解：实际项目仍要看框架、权限、数据脱敏和性能开销。

OpenTelemetry 的直觉

OpenTelemetry 像物流系统中的运单标准。包裹经过仓库、分拣中心和配送站时，每一站都留下时间、地点和状态。AI 应用中的一次请求也会经过模型调用、工具调用、检索、解析和业务

² 视频画面时间区间：00:03:20-00:04:15。若为裁剪图，时间区间仍对应原视频画面。

逻辑，trace/span 就是这些节点的结构化运单。

一个 trace 可以用教学记号写成：

$$T = (span_1, span_2, \dots, span_n)$$

其中， T 表示一次端到端运行或请求； $span_i$ 表示第 i 个调用片段，例如一次 LLM 调用、一次工具调用、一次检索步骤或一次解析步骤； n 表示该 trace 中 span 的数量。这个公式是笔记为了教学添加的记号，并非 Dat 在演讲中给出的数学定义。

2.2 Trace、span 与 session：从调用片段到状态过程

Trace / 调用轨迹记录一次端到端运行；span / 调用片段记录其中一个有时间边界的步骤。Dat 把 trace 和 span 称为 agent 行为的审计记录，因为它们回答了“系统实际执行了什么”。对生产系统来说，这比事后阅读散乱日志更有诊断价值。

Dat 还提出 sessions / 会话。Session 面向多轮状态：用户和 agent 之间的往返对话、agent 内部状态的变化、一次长任务中的多个 run，都可能被组织成 session。很多企业用户并不总是关心某个底层 tool call 的细节，他们更关心“终端用户是否满意”“所有问题是否被回答”。这个问题的作用域已经超出单个 span，需要 session 层面的观察。

dashboard 只是表面

dashboard 很有用，因为它让人快速看到延迟、错误、路径和质量分布。真正支撑诊断的是底层 trace、span 和 session 数据。只有图表截图会让团队看到现象；结构化遥测才让团队追到根因。

2.3 Branch、path 与 distributional view：观察许多次运行

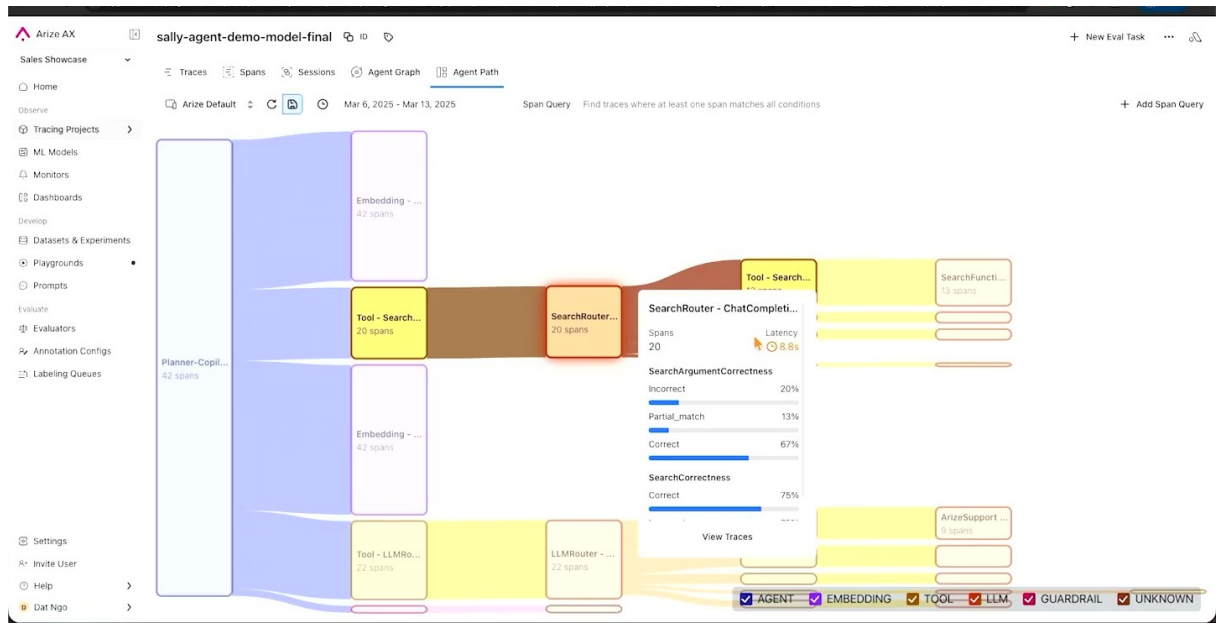


图 4: Agent Path 视图把许多次运行汇总成分支流量、组件耗时和局部 eval 结果，帮助团队观察路径分布。³

智能体系统常常存在分支 (*branch*) 和路径 (*path*)。同一个任务可能走检索路径，也可能走工具调用路径；可能进入循环，也可能提前结束。Dat 展示的 Arize AX 视图强调：团队可以看单次 trace，也可以看所有运行实例上的分布式视角 (*distributional view*)。

分布式视角回答的问题更加接近生产管理：多少流量进入某一条分支？某条分支是否引起显著延迟？某类路径上的 eval signal 是否下降？如果把单条 trace 比作一段行车记录仪，distributional view 就像城市交通热力图：单车视角能解释一次事故，热力图能发现系统性拥堵点。

³ 视频画面时间区间：00:05:00-00:05:45。若为裁剪图，时间区间仍对应原视频画面。

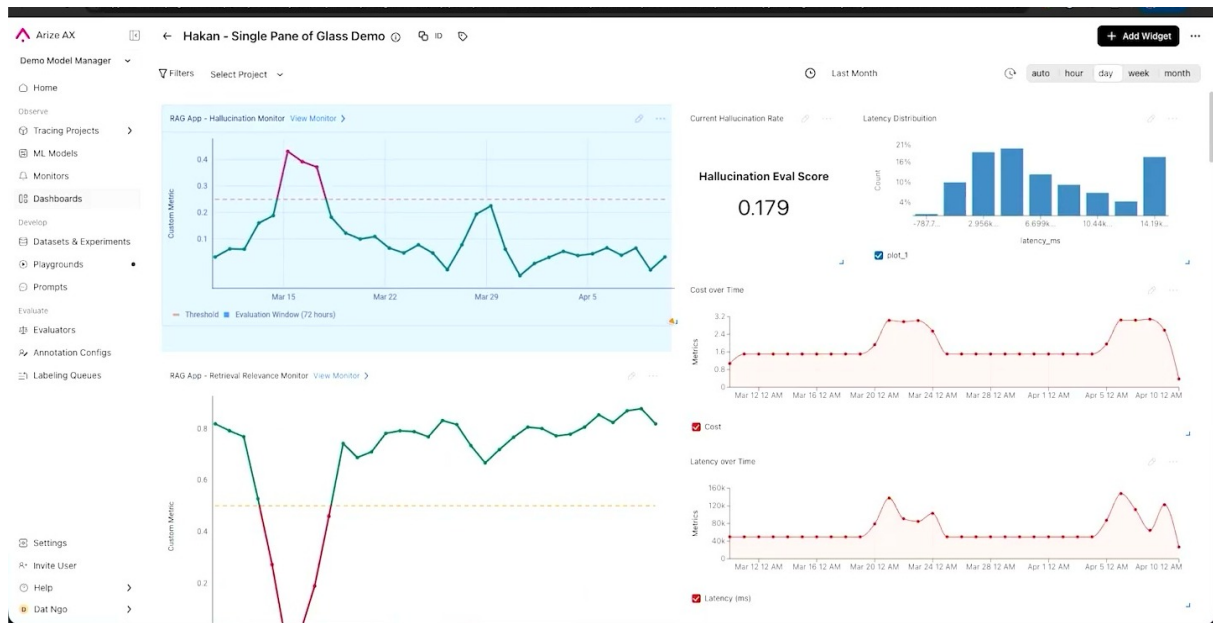


图 5: Dashboard 将 hallucination、retrieval relevance、latency 和 cost 等信号放在同一生产视图中。⁴

Dat 还提到 analytics 并没有消失。企业团队仍然需要实时构建视图，观察 agent 的整体状态。这类视图可以展示幻觉、检索、延迟、成本等维度；但这些演示数据只应作为说明 workflow 的例子，不能被当成普遍 benchmark。

2.4 Root cause: 从路径异常追到因果结构

W05 中的根因示例很关键。Dat 说，有时某一路径信号下降，根因可能是两个组件顺序错了：系统先做了 B，随后才做 A，可 B 依赖 A。LLM 的调用决策与业务依赖顺序不匹配，修复方式可能就是给模型更多上下文，明确“做这个动作之前要先做那个动作”。

这个例子说明可观测性并不只是在界面里看漂亮图。它要把“某条路径质量下降”连接到“哪两个组件顺序错误”这种可修复事实。对工程师来说，root cause / 根因就是从症状走向机制；对产品经理和业务专家来说，根因解释了为什么某类用户体验会崩掉。

可观测性的 zoom ladder

本讲可以把可观测性理解为一个缩放梯：span 看组件输入输出，trace 看一次运行，trajectory 看步骤路径，session 看多轮状态，distributional view 看许多运行上的分布规律。层级越高，越能回答产品和业务问题；层级越低，越利于定位工程根因。

2.5 从本章进入下一章

可观测性解决了“看见”的问题。看到 trace、span、session 和路径分布后，团队还需要回答“这些行为质量如何”。下一章进入评估信号：LLM 裁判、确定性检查、标注、用户反馈和业务指标如何把遥测证据转成可行动的 signal。

⁴视频画面时间区间：00:06:20-00:07:10。若为裁剪图，时间区间仍对应原视频画面。

2.6 本章小结

本章说明了第一根支柱：*observability* 通过 OpenTelemetry、auto instrumenter、traces、spans、sessions、branch/path 视图和 dashboard，把智能体系统的内部行为变成可审计证据。Dat 的重点在于：团队既要能看单次运行，也要能看许多运行上的路径分布和延迟分布；既要表面指标，也要追到 root cause。没有这层证据，后续 evals 与 experiments 很容易变成盲调。

3 评价信号：从 LLM-as-a-Judge 到业务指标

评估的核心任务是从系统行为中抽取可信信号。Dat 在 W05-W07 中把 signal 分为五类：LLM 作为裁判、代码/逻辑评估、标注、用户反馈和业务指标。这个分类的价值在于，它提醒团队不要把“评估”缩小成单一打分器；生产 AI 的质量判断通常来自多种证据的组合。

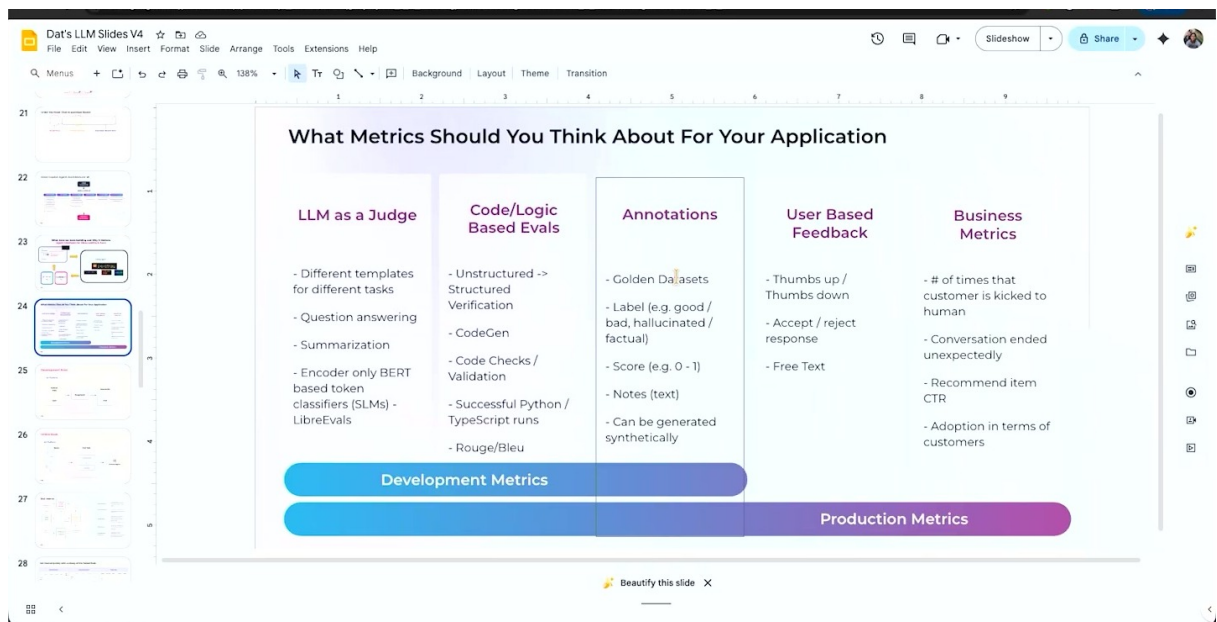


图 6: Dat 的五类 signal slide: LLM judge、code/logic evals、annotations、user feedback 和 business metrics。⁵

five flavors of signal

本讲中的五类信号可以概括为：*LLM as a judge*、代码或逻辑评估、人工/专家标注、用户反馈、业务指标。它们分别回答灵活语义判断、确定性约束、可信参考、真实体验和商业结果问题。

3.1 LLM as a Judge: 灵活但需要校准

LLM as a judge 指用一个 LLM 根据标准、rubric 或提示词去评分、分类或批评另一个模型/agent 的输出。它的优势是灵活：可以判断相关性、语气、完整性、安全性、是否遵循指令等难以用简单规则覆盖的问题。它的弱点也很现实：judge 可能不稳定、偏向某种措辞、被输入诱导，或在不同领域中误读质量标准。

⁵ 视频画面时间区间：00:07:31-00:09:04。若为裁剪图，时间区间仍对应原视频画面。

Dat 提到 *golden data sets*，即黄金数据集。黄金数据集的价值在于第三列质量标签来自可信的人或领域专家。团队可以让 LLM judge 去近似这些可信标签：如果 judge 的判断经常接近专家标签，它才更适合在大规模数据上提供自动化信号。

黄金数据集的作用

黄金数据集像考试中的标准答案与阅卷细则。它的作用是为 judge 和 eval workflow 提供校准点，覆盖范围只需围绕关键任务和高风险场景展开。没有可信参考，LLM judge 的分数很容易变成“看起来很精确的主观意见”。

LLM judge 的噪声

LLM judge 的输出是有用信号，却需要校准、抽检和版本管理。更换 judge 模型、rubric 或提示词会改变分数分布。生产团队应记录 judge 版本，并将重要评估与可信标签或人工复核连接起来。

3.2 确定性评估：便宜、清晰、可自动化

Dat 强调，团队不必总是使用 LLM call 或人类判断。确定性（determinism）在很多场景里非常有价值。典型例子是从自然语言段落生成 JSON payload：评估可以检查 JSON 是否有效、schema 是否匹配、必要字段是否非空。这类代码/逻辑评估的判断边界清楚，成本低，速度快。

确定性 eval 类似软件工程中的单元测试。它无法判断答案是否有洞察力，但可以非常稳定地挡住结构错误。例如，一个业务流程要求输出必须包含 `customer_id`、`risk_level` 和 `next_action`，那么字段缺失或类型错误就应被立即标记。

代码 / 逻辑评估的适用场景

当质量标准可以写成规则、schema、正则、执行结果或数值阈值时，优先考虑确定性 eval。它们通常更便宜、更快、更可解释，也更适合接入 CI 或在线监控。

3.3 人、用户与业务：质量定义不只属于工程团队

Dat 特别提醒“不能忘记人”。人类信号包括领域专家标注、终端用户反馈、产品经理定义的体验标准等。用户反馈可以是 thumbs up/down、接受/拒绝、自由文本评价、会话中断或人工转接。业务指标则落到更硬的结果：赚钱、节省成本、节省时间。

这带来一个组织层面的重点：技术建设者与领域专家必须共同拥有 eval。工程师擅长框架、自动化和程序化接入；产品经理、业务专家和 subject matter experts 更理解 AI 体验应该是什么样。Dat 的说法是，能写代码的人应把编码工作做好，懂领域的人应在领域质量判断上发挥作用。

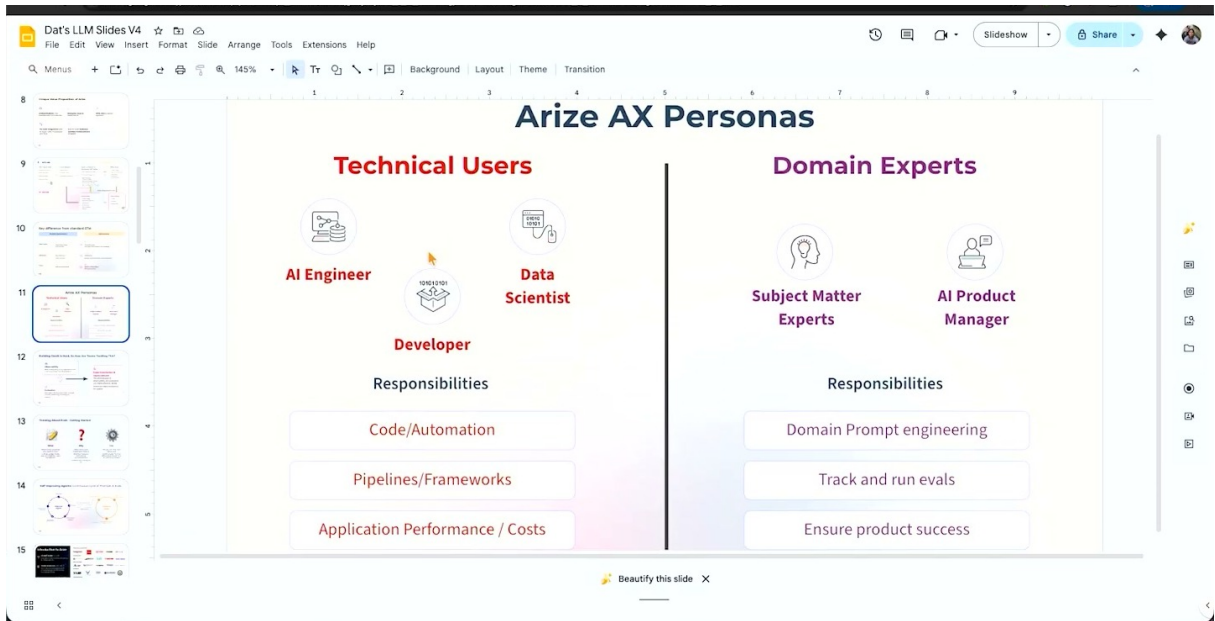


图 7: Arize AX Personas slide 展示了技术用户与领域专家在 eval workflow 中的不同职责。⁶

在 Arize 的演示中，这种分工体现为两种入口：非技术用户可以通过界面选择模型、套用模板或自定义 eval；技术用户可以用程序化方式挂载 eval 并运行。这里的产品能力仍应写成 Dat/Arize 的演示主张，后续 review agent 可以结合画面确认 UI 细节。

3.4 用复合分数作为教学脚手架

为了理解多信号如何组合，可以用一个简单的教学公式：

$$Q = \sum_{i=1}^k w_i s_i$$

其中， Q 表示一个综合质量分数； s_i 表示第 i 个评估信号，例如 judge 分数、JSON 合规分数、用户反馈分数或业务指标； w_i 表示该信号的权重； k 表示纳入组合的 eval 数量。这个公式只是帮助读者理解“多信号组合”的概念。真实系统需要校准、阈值、分布监控和人工解释，不能把加权和当成万能答案。

业务指标与开发指标的错位

开发指标 (*development metrics*) 能帮助团队快速迭代，例如准确率、格式合规、judge 分数。生产指标 (*production metrics*) 反映真实结果，例如转化率、人工转接率、节省时间和客户满意度。开发指标变好时，业务指标仍可能没有改善。

3.5 从本章进入下一章

本章回答了“用什么方法抽取 signal”。下一章要进一步拆开另一个问题：eval 究竟评估哪个作用域？同样是 LLM judge，它可以评估一个 span 的输出，也可以评估整个 session 的用户体验。方法与作用域混在一起，会让团队误判系统质量。

⁶视频画面时间区间：00:09:01-00:10:34。若为裁剪图，时间区间仍对应原视频画面。

3.6 本章小结

本章的主张是：evals 是信号提取系统。Dat 给出的五类 signal 分别覆盖语义判断、确定性规则、可信标注、真实用户体验和业务结果。LLM judge 提供灵活性，黄金数据集提供校准依据，确定性 eval 提供低成本约束，人类和业务指标提供最终语境。评估系统的质量来自信号组合和校准过程，不能只依赖某一个看起来方便的打分器。

评分方法	Span / Trace	Trajectory	Session
LLM judge	评价一次模型回复是否完整、相关、安全	判断一条路径是否符合任务目标	判断多轮会话是否解决用户问题
确定性规则	JSON、schema、必填字段、执行成功	工具顺序、依赖顺序、循环检测	SLA、超时、人工转接触发
人工/专家标签	对关键样本标注质量	审查复杂业务流程是否正确	审查用户体验和领域标准
业务指标	单步成本、延迟、错误率	流程完成率、失败路径比例	满意度、留存、人工介入率

图 8: 评估方法与评估作用域是两条轴：同一种方法可以用于不同范围，同一范围也可以组合多种方法。

4 评价作用域：Span、Multi-span、Trajectory、Session

评价作用域回答的是“系统的哪一部分正在被评分”。Dat 在 W07-W09 中把 eval 的范围从单个组件扩展到多个组件、完整轨迹和会话级状态。这里最容易混淆的是评估方法与评估作用域：*LLM as a judge* 是一种方法，*span eval*、*multi-span eval*、*trajectory evals* 和 *session level eval* 是评估覆盖的系统范围。

不要混淆方法与作用域

“用 LLM judge 打分”说明的是评分方法；“评估一个 span、多个 span、整条 trajectory 或整个 session”说明的是评分对象。两者可以组合，例如用 LLM judge 做 session level eval，也可以用确定性规则做 span eval。

4.1 Span eval: 看一个组件的输入与输出

span eval 是最容易理解的层级：看一个 LLM call、工具调用、检索步骤或解析组件的输入输出。它类似检查流水线上的一个工位：原料是什么，产出是什么，是否符合该工位的规格。

Span 级评估适合回答局部问题：模型是否遵循输出格式？检索结果是否包含所需字段？某个解析器是否生成有效 JSON？工具调用参数是否完整？这些问题范围小，因果链短，因此通常更易自动化。

Span 的边界

Span 是 trace 中一个有时间边界的调用片段。它既有输入输出，也有延迟、错误、成本和元数据。把 span 设计清楚，后续 eval、debugging 和 dashboard 才有稳定颗粒度。

4.2 Multi-span eval: 跨组件检查数据传递

multi-span eval 适用于一个评估需要多个组件数据的场景。Dat 举的例子是 agent 之间如何传递数据。如果要判断数据是否正确从一个 agent 传给另一个 agent，单看某个 span 不够，需要把多个 span 的输入输出放在一起检查。

这类 eval 的复杂度更高，因为它要处理跨组件一致性。例如，检索 span 找到的合同编号是否被后续摘要 span 正确引用？工具调用返回的金额是否在最终回复中被错误改写？上游分类结果是否影响了下游决策路径？这些问题都需要 multi-span 视角。

4.3 Trajectory eval: 检查步骤顺序和业务过程

trajectory evals 关注完整路径。Dat 的根因示例可以自然连接到这里：系统先做 B 再做 A，但 B 对 A 有依赖，说明轨迹顺序错了。Trajectory eval 评估的不只是单点输出质量，还包括步骤是否按正确顺序推进，是否漏掉关键动作，是否进入无意义循环，是否完成业务过程。

可以把 trajectory 视为：

$$P = (step_1, step_2, \dots, step_n)$$

其中， P 表示一次 agent 轨迹； $step_i$ 表示第 i 个决策或动作； n 表示轨迹长度。若业务规则要求 $step_a$ 必须早于 $step_b$ ，那么 eval 就要检查这种依赖顺序。这个公式是笔记添加的教学记号，用于帮助读者理解轨迹评估。

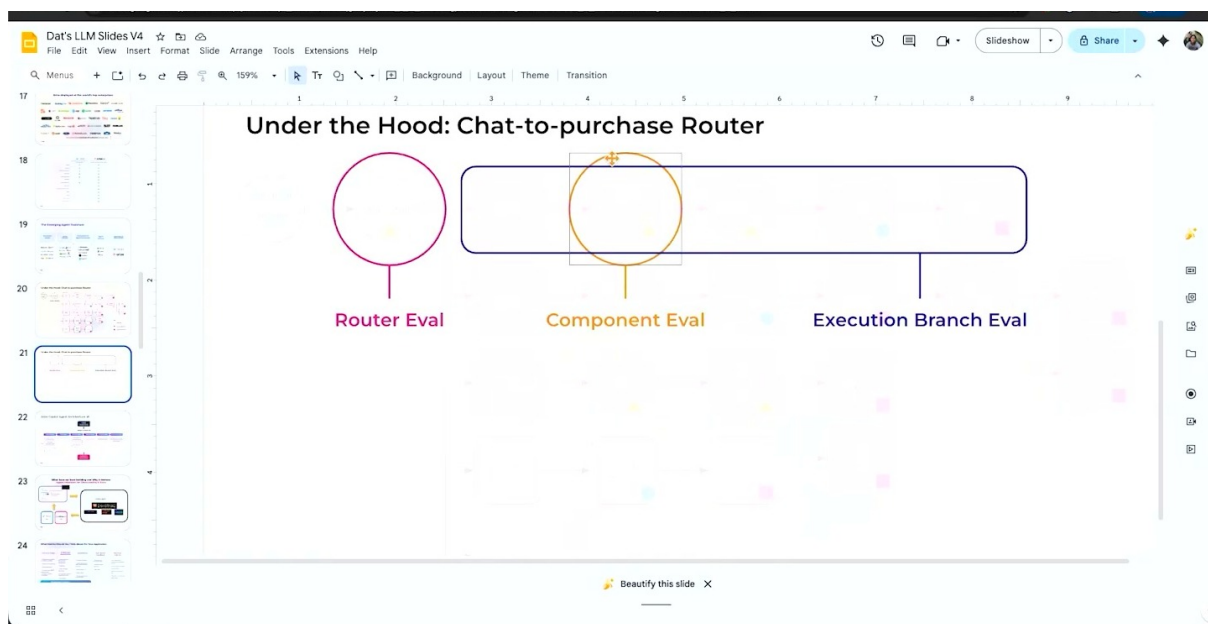


图 9: Chat-to-purchase router 的示意图把 router eval、component eval 和 execution branch eval 放在同一条流程上，直观呈现不同评估作用域。⁷

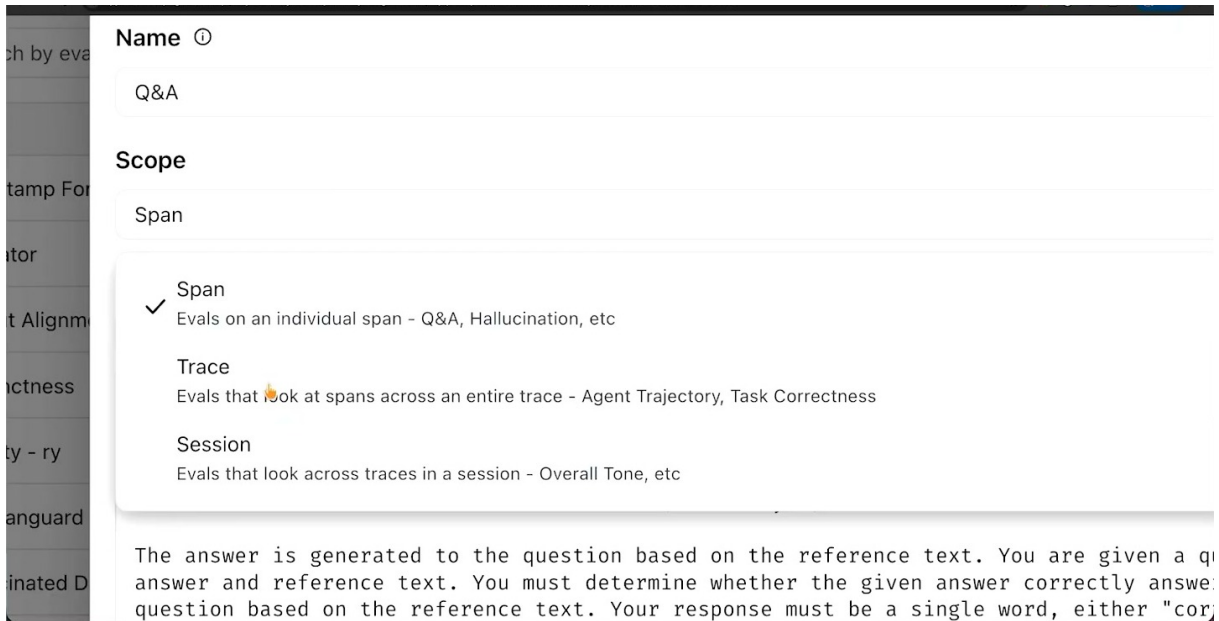


图 10: Create Evaluator 界面把 scope 分成 Span、Trace 和 Session，说明 eval 方法与 eval 作用域需要分开理解。⁸

eval scope ladder

评估作用域可以看成一架缩放梯：span eval 检查单个组件，multi-span eval 检查组件之间的数据传递，trajectory eval 检查完整步骤路径，session level eval 检查多轮状态和用户体验。粒度越细，越利于定位局部错误；范围越大，越贴近真实业务体验。

4.4 Session level eval: 评估状态机与用户体验

session level eval 把会话看作一个状态机。Dat 的例子包括：用户是否曾经感到沮丧？系统是否回答了用户所有问题？这些问题无法从单个 span 中可靠判断，因为用户体验往往跨越多个 turn。

一个 session 可以用教学记号写成：

$$S = (turn_1, turn_2, \dots, turn_m)$$

其中， S 表示一个会话； $turn_j$ 表示第 j 个用户或系统轮次； m 表示总轮次数。会话级 eval 关注状态随时间如何变化，例如用户问题是否被逐步解决、是否反复追问、是否从满意转为困惑。

4.5 Minimal set of evals: 评估越多，成本也越高

Dat 在 W08-W09 中给出一个非常务实的提醒：能评估某件事，不代表总要评估它。团队要寻找 *minimal set of evals*，即能判断应用是否按预期工作的最小有用 eval 集合。原因很直接：eval 有成本，包括 LLM 调用成本、延迟、工程维护、标注成本、阈值调参和误报处理。

可以用集合记号表达这个想法：

$$E = \{e_1, e_2, \dots, e_k\}$$

⁷ 视频画面时间区间：00:11:30-00:11:58。若为裁剪图，时间区间仍对应原视频画面。

⁸ 视频画面时间区间：00:10:45-00:11:15。若为裁剪图，时间区间仍对应原视频画面。

其中, E 表示当前选择的 eval 集; e_i 表示第 i 个 eval; k 表示 eval 数量。团队希望 E 足以覆盖关键风险, 同时避免把低价值检查堆满系统。

最小有用 eval 集

好的 eval 组合应覆盖关键失败模式、关键业务目标和关键回归风险。它的目标是提供足够 signal, 让团队能判断系统是否按预期工作。eval 数量本身不能代表成熟度。

eval 成本

每个 eval 都会带来计算成本、延迟、维护成本和解释成本。LLM judge 还会引入模型版本、提示词、rubric 和校准数据的管理成本。团队应把 eval 当成生产资产管理, 避免把它当作一次性脚本。

4.6 从本章进入下一章

作用域清楚之后, eval 才能进入实验流程。下一章会把 trace、dataset 和 input-output pairs 连接起来, 说明团队如何比较提示词、模型、编排和配置改动, 并用 evals 观察隐藏回归。

4.7 本章小结

本章把 eval 拆成两个正交问题: 评分方法与评分作用域。Dat 展示的作用域从 span 到 multi-span, 再到 trajectory 和 session。Span 适合局部组件, multi-span 适合跨组件数据传递, trajectory 适合业务步骤顺序, session 适合多轮状态和用户体验。团队还要记住 *minimal set of evals*: 评估要覆盖关键风险, 同时控制成本与复杂度。

5 实验与改进: 把观察到的问题变成可验证的变化

实验与改进的核心任务, 是把“观察到的问题”变成“可比较的候选变化”。Dat 在 W09-W10 中说明, 团队可以从 traces 出发, 把低质量信号对应的样本收集成 dataset; 也可以直接上传 input-output pairs。随后, 团队针对 prompts、models、orchestration 和 configurations 做实验, 观察 evals 与指标如何变化。

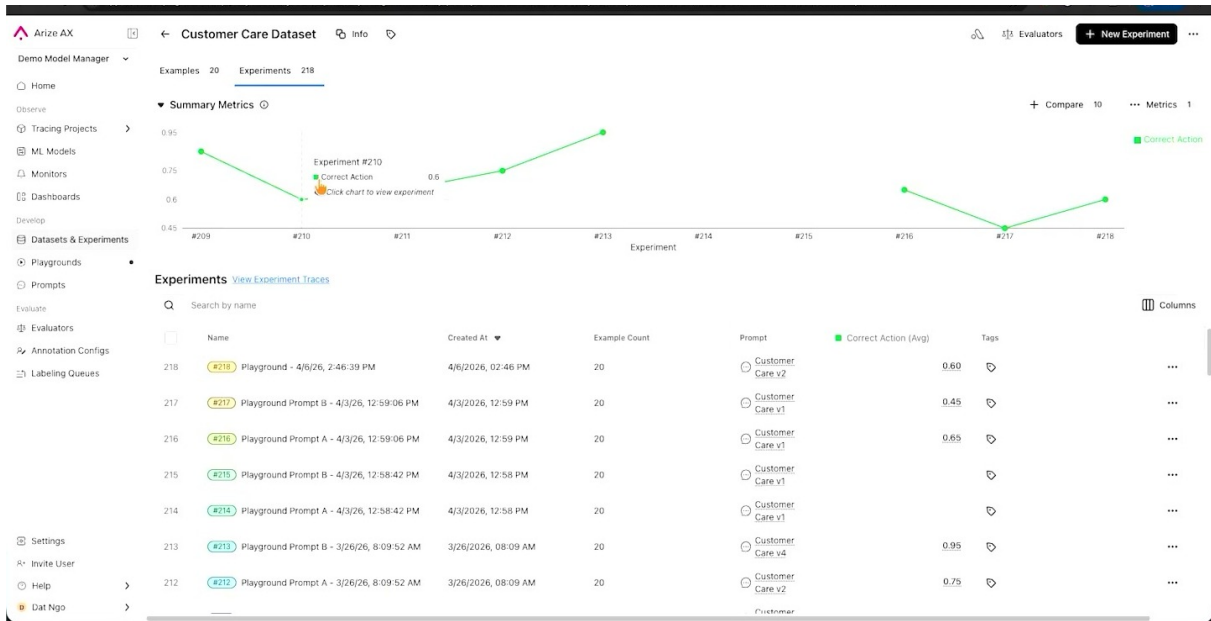


图 11: Customer Care Dataset 的 experiments 视图展示了实验编号、样本量、prompt 版本和 Correct Action 指标。⁹

实验的定义

在本讲语境中，实验（*experimentation*）是对候选改动的受控比较。候选改动包括 *changes to prompts*, *changes to models*, *changes to orchestration*, *changes to configurations*。实验的证据来自 traces、datasets、evals 和生产指标。

5.1 从 traces 到 dataset：把失败样本收集起来

Dat 说，并非每个团队都从 traces 开始；如果已有 traces，就可以找到信号较差、缺失信息或行为异常的样本，把它们收集到 dataset。这个动作很关键：trace 是运行证据，dataset 是可重复比较的实验材料。

input-output pairs / 输入输出样本对可以写成：

$$(x, y)$$

其中， x 表示应用输入、用户问题或任务上下文； y 表示模型或 agent 输出。若存在可信参考答案或人工标签，可以扩展为：

$$(x, y, r)$$

其中， r 表示 trusted reference、human label 或 golden answer。这个记号用于解释 dataset 结构，方便后续理解 eval 如何比较输出和参考。

Dataset 与日志堆不同

dataset 是为了重复实验而组织的数据。它通常包含行与列，能支持同一批样本在不同 prompt、model、orchestration 或 configuration 下反复运行。日志记录过去发生了什么；dataset 支持未

⁹ 视频画面时间区间：00:12:15–00:13:15。若为裁剪图，时间区间仍对应原视频画面。

来做可比较实验。

5.2 四类常见改动

第一类是 prompt 改动。团队可能增加约束、示例、工具使用说明、输出格式要求或依赖顺序提醒。例如 W05 的 B 先于 A 的问题，可能通过上下文说明来修复：在执行 B 前必须先完成 A。

第二类是 model 改动。更换模型可能提升语义判断、推理能力或工具使用质量，也可能增加成本和延迟。模型切换尤其需要评估边界案例，因为高平均分可能掩盖少数关键场景的退化。

第三类是 orchestration / 编排改动。编排决定调用哪个工具、何时检索、何时让 agent 分支、是否允许循环、怎样把数据从一个组件传给另一个组件。对智能体系统来说，编排质量常常决定 trajectory 是否合理。

第四类是 configuration / 配置改动。温度、最大 token、检索阈值、重试策略、超时设置、工具权限和缓存策略都可能影响质量、成本与稳定性。配置看似细节，生产影响往往很大。

5.3 实验比较：关注指标变化与隐藏回归

为了把“改动后好像更好”转成可检查结论，可以用指标变化表达：

$$\Delta M = M_{\text{new}} - M_{\text{base}}$$

其中， M_{new} 表示新版本的某个指标， M_{base} 表示基线版本的同一指标， ΔM 表示改动后的变化。若 M 是准确率，正值通常是好事；若 M 是延迟或成本，正值可能代表风险。每个指标的方向必须明确说明。

局部修复与隐藏回归

一个 prompt fix 可能让某个 eval 上升，同时让延迟、成本、拒答率、事实性或业务转化恶化。实验应同时观察核心质量指标、保护性指标和业务指标。只看修复目标会让团队漏掉旁路损伤。

改动类型	可能改善	必须守住的风险
Prompt	指令遵循、格式、依赖顺序	冗长、成本上升、过度约束
Model	推理、语义理解、工具选择	价格、延迟、版本迁移风险
Orchestration	路径正确性、工具协同	死循环、遗漏步骤、权限扩大
Configuration	稳定性、吞吐、检索质量	阈值漂移、超时、缓存污染

5.4 UI 与程序化实验

Dat 提到 Arize 允许团队在 UI 或程序化环境中测试这些改动。UI 适合探索、协作和让非技术角色参与；程序化入口适合自动化、批量实验、CI 集成和 coding-agent workflow。二者服务于同一个工程目标：用相同 dataset 和 evals 比较候选版本。

这里有一个常被低估的组织问题：实验结果需要被解释。若新版本在 judge 分数上升、确定性格式检查稳定、业务指标不变、延迟轻微上升，团队要根据产品目标决定是否接受。实验平台提供证据，

决策仍需要工程、产品和领域角色共同判断。

5.5 从本章进入下一章

本章讲的是手动或程序化的实验闭环。下一章进入 Dat 对未来方向的判断: *software will compress*, 团队不会长期停留在 dashboard、按钮和手动操作里; CLI、tools、skills、coding agents 和 Alex 会把观测、评估与实验进一步自动化。

5.6 本章小结

本章的结论是: 实验把可观测性和评估转化为改进机制。团队可以从 traces 收集失败样本, 也可以上传 input-output pairs, 随后围绕 prompts、models、orchestration 和 configurations 做受控比较。实验的重点在于同时观察目标指标和防护指标, 尤其要防止隐藏 regressions。没有实验, 团队只是调参; 有了 dataset、evals 和指标比较, 改动才进入工程闭环。

6 自动化飞轮: 让 AI 参与观测、评价与改进

Dat 在演讲最后把视角从“人如何使用平台”推进到“AI 如何参与这个平台”。他的判断是 *software will compress*: 构建和定制软件会变得更易, 团队不应长期依赖 dashboard、按钮和手动排查。Arize 的演示主张是, 把 observability、evals、experimentation and improvement 的底层能力暴露给 CLI、tools、skills 和 coding agents, 再由 Alex 这类助手参与分析、计划和生成 eval。

自动化飞轮

自动化的目标是缩短闭环: 系统记录 traces 与 sessions, evals 抽取 signal, 实验比较候选变化, AI 助手根据上下文发现问题并提出或创建新的 eval。Dat 用 *automate you out of this process* 表达这一愿景: 让团队从重复操作中退出, 把注意力放在目标、风险和治理上。

6.1 CLI、tools、skills: 把平台能力交给 coding agent

Dat 提到, Arize 把 primitives 暴露为 CLI、tools and skills。这句话对 AI 工程团队很重要: 一旦可观测性和 eval 平台只有网页按钮, coding agent 很难稳定调用; 一旦能力变成命令行、工具和技能, agent 就可以把它们纳入开发循环。

一个可行的工程图景是: 开发者在 Codex、Claude Code 或类似 coding agent 中改动 prompt; agent 调用实验工具跑同一批 dataset; eval 工具返回质量、延迟和成本变化; agent 根据失败样本生成候选修复; 人类审核高风险变更。Dat 没有承诺所有团队今天都能完全自动完成这件事, 他表达的是 Arize 的产品方向和自动化愿景。

coding-agent integration 的意义

当 observability/eval/experiment 能被 CLI 和工具调用时, 它们就能进入开发者日常工作流: 本地调试、PR 检查、回归套件、批量实验和自动报告。网页 UI 适合人类探索; 工具接口适合自动化和可复现流程。

6.2 Alex: 在 trace 与 eval 上工作的助手

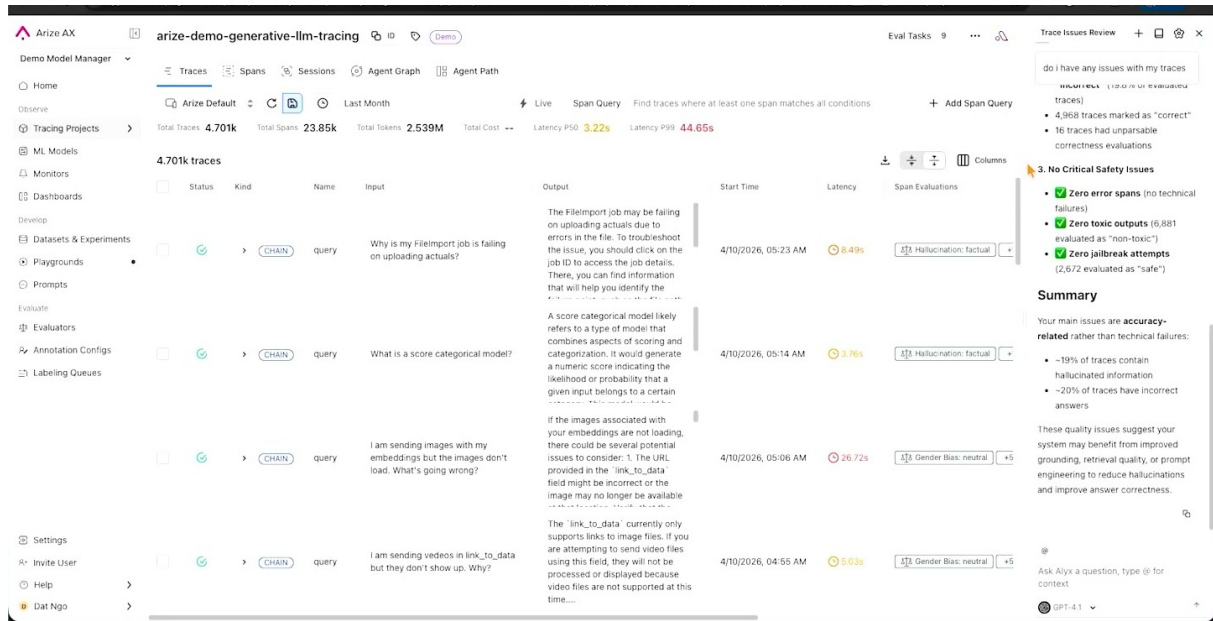


图 12: Alex/Trace Issues Review 面板基于 traces 总结 hallucination、incorrect answer 和 safety 等问题。¹⁰

Dat 展示的 Alex 是 Arize 演示中的 AI 助手。因为平台拥有 traces、hooks 和相关数据，Alex 可以被询问“我的应用有没有问题”。在演示描述中，Alex 会进入系统、规划并运行任务，帮助发现 high latency、errors detected 等问题。

这里的关键点是上下文。普通聊天机器人没有 production traces；Alex 的价值主张来自它能够访问平台中的遥测和 eval 资料。可以把它类比为带着完整病历、化验单和治疗记录的住院医师，而非只听患者一句描述就下判断的外部顾问。

自动化不等于免治理

AI 助手可以发现异常、生成 eval 或提出实验，但生产团队仍要管理权限、数据隐私、误报、成本和变更审批。越多能力交给 agent，越需要审计日志、权限边界和人工复核策略。

6.3 动态 eval: 让系统根据变化提出新的检查

Dat 在 W11 中提出一个更激进的目标：团队不一定总要自己选择 eval。AI 应该有上下文：这里有 traces，这里发生了变化，那么它可以动态创建 eval，或在发现新变化时提出新的评估。这个想法与前文的 *minimal set of evals* 并不冲突；它说明最小集合可以随着系统变化而更新。

例如，系统近期引入了新的工具调用路径，Alex 可能建议增加 trajectory eval，检查新路径是否遵守依赖顺序；如果线上出现高延迟，它可能建议增加 latency guardrail；如果用户反馈集中在“没有回答完整问题”，它可能建议 session level eval。这里仍需人工判断 eval 是否值得长期保留。

¹⁰ 视频画面时间区间：00:14:05–00:14:55。若为裁剪图，时间区间仍对应原视频画面。

动态 eval 的边界

动态 eval 适合发现新风险和补齐监控盲区。它的长期价值取决于校准、去重、成本控制和治理。自动生成 eval 后，团队仍要判断它是否覆盖关键风险、是否误报过多、是否应进入正式回归套件。

6.4 Phoenix 与 Arize AX：两个产品定位

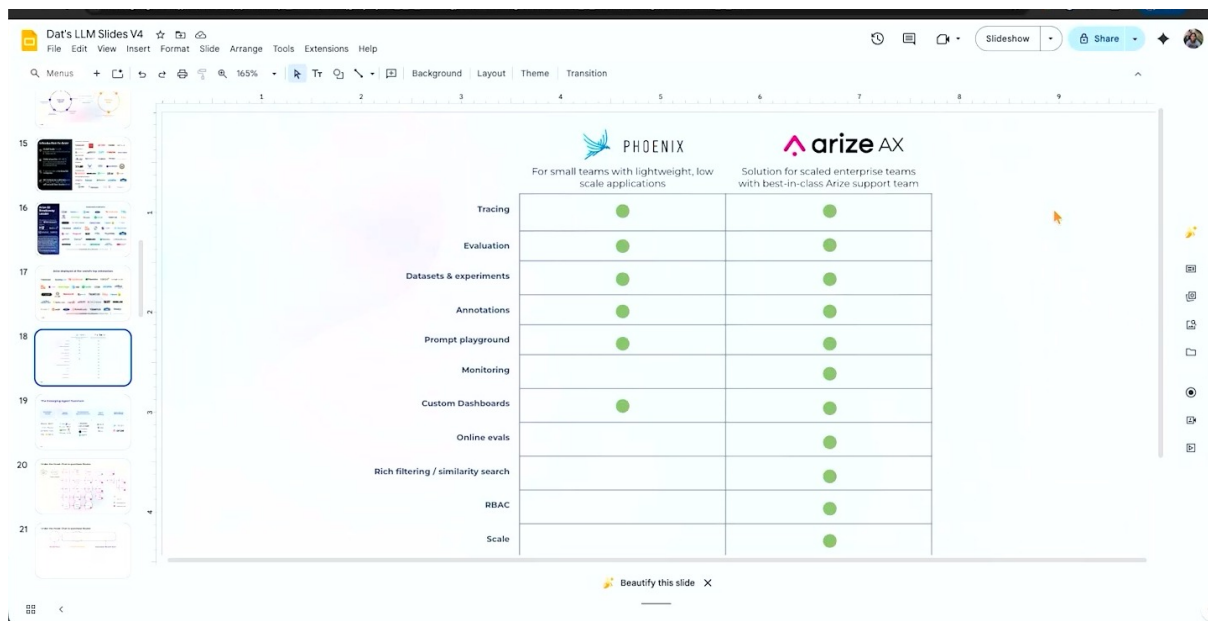


图 13: Phoenix 与 Arize AX 的能力矩阵体现了 Dat 在结尾给出的产品定位：轻量开源与企业平台。

11

演讲收尾时，Dat 介绍 Arize 当前有两个产品。面向 engineering-first 团队的是 *Phoenix*，他称其为开源、single container、可本地部署，并且不需要 Kubernetes layer。面向大型企业的是 *Arize AX*，Dat 将其描述为给最大规模企业使用的产品，并在演示中提及 Uber、Booking 和 Reddit 等客户语境。

这些内容应写成产品定位说明。*Phoenix* 更像轻量、本地、工程优先的入口；*Arize AX* 更像企业级平台，承载更大规模、更多治理和平台能力。笔记不应从演讲推导出未经验证的市场比较，只需忠实呈现 Dat 在收尾部分的产品分层。

Phoenix 与 AX 的读法

Phoenix 可以被理解为工程团队开始 tracing、evals 和本地实验的轻量入口；*Arize AX* 是 Dat 在演讲中展示的企业平台语境。两者都是 Arize 产品线中的组成部分，具体能力和授权边界需要后续以官方文档或实际部署为准。

¹¹ 视频画面时间区间：00:15:30-00:16:05。若为裁剪图，时间区间仍对应原视频画面。

6.5 从本章进入下一章

自动化飞轮把前六章串起来：observability 提供上下文，evals 提供信号，experimentation 提供比较，Alex 与工具接口负责缩短人的重复操作。下一章会把这套演示系统上升为生产治理模式，给不同角色总结实践路径和风险清单。

6.6 本章小结

本章说明 Dat 对未来工作流的判断：*software will compress*，观测、评估和实验会越来越多地通过 CLI、tools、skills、coding agents 和 Alex 参与自动化。自动化的目标是让系统更快发现问题、创建 eval、运行实验并暴露回归风险，同时保留必要的工程判断。Phoenix 与 Arize AX 则代表 Dat 在演讲中给出的产品分层：一个偏工程优先和轻量部署，一个偏大型企业平台。

7 总结与延伸：从演示系统到生产治理

这场短讲的最终价值不在于某个 dashboard 或某个产品按钮，而在于它把生产 GenAI 的治理路径压缩成一个可执行闭环：Observe、Evaluate、Experiment、Improve。Dat 的 closing emphasis 是自动化：AI 应该能基于 traces 和变化动态创建 eval，发现 latency、errors 等问题，并帮助团队推动改进。这个愿景的前提是前面六章已经建立的工程资产：遥测、信号、作用域、dataset、实验和工具接口。

全篇总命题

生产 LLM/agent 系统的治理逻辑是：先记录事实，再提炼信号，再比较改动，最后把有效改动纳入系统。没有 telemetry，团队缺少事实；没有 evals，团队缺少判断；没有 experiments，团队缺少因果比较；没有 automation，闭环速度会跟不上系统变化。

7.1 四层治理模型

第一层是事实层。事实层由 OpenTelemetry、traces、spans、sessions 和 agent graph 构成。它回答“系统实际做了什么”。在这层里，团队要关心埋点覆盖、数据脱敏、元数据命名、trace/span 边界和 session 归属。

第二层是信号层。信号层由 LLM judge、确定性 eval、标注、用户反馈和业务指标构成。它回答“这些行为是否可接受”。在这层里，团队要关心 judge 校准、黄金数据集、误报、漏报、成本和开发指标与生产指标的连接。

第三层是实验层。实验层由 datasets、input-output pairs、candidate changes 和 regression guardrails 构成。它回答“某个改动是否真的改善了系统”。在这层里，团队要同时看目标指标和保护性指标，尤其是质量、延迟、成本、安全和业务结果之间的相互影响。

第四层是自动化层。自动化层由 CLI、tools、skills、coding agents 和 Alex 这类助手构成。它回答“如何让闭环更快运行”。在这层里，团队要把权限、审计、变更审批和人工复核设计清楚。

7.2 给不同角色的实践路线

AI 工程师首先要把 trace/span/session 的边界设计好。最常见的失败来自底层数据无法回答问题：工具调用参数没有记录、检索结果没有关联到最终回答、session ID 混乱、错误和延迟缺少上下文。dashboard 的美观程度解决不了这些数据模型问题。工程师还要把关键 eval 接入自动化流程，让 prompt、model、orchestration 和 configuration 的改动都能被重复比较。

产品经理应参与定义体验质量。Dat 在演讲中强调 subject matter experts 和 product managers 理解 AI experience 应该是什么样。产品经理需要把“用户满意”“问题回答完整”“业务流程完成”这些模糊目标转成可评估标准，并决定哪些指标代表真实价值。

领域专家应参与黄金数据集和标注。LLM judge 的质量上限很大程度取决于可信参考。没有领域专家，团队容易用通用语言流畅度替代真实业务正确性。对医疗、金融、法律、客服、供应链等领域尤其如此。

技术领导应关注治理边界。自动化 eval 和 Alex 这类助手能提高速度，也会扩大权限和影响面。领导层需要规定哪些变更可以自动运行，哪些需要人工批准，哪些数据不能进入外部模型，哪些指标触发发布冻结。

从演示到生产的差距

演示系统展示的是 workflow 的可能形态。生产系统还要处理数据治理、权限、PII 脱敏、成本预算、模型版本迁移、线上回滚、审计留痕和跨团队责任边界。这些内容没有在短讲中展开，却是落地时绕不开的未知风险。

7.3 核心风险清单

第一，过度相信单一信号。一个 LLM judge 分数、一个 dashboard 图表或一个用户反馈入口都无法代表全部质量。团队应把多个信号组合起来，并清楚每个信号的盲区。

第二，把 eval 当作一次性资产。eval 会老化：模型版本变化、产品目标变化、用户群变化、攻击方式变化，都会让旧 eval 失效。动态 eval 的价值就在于提醒团队风险空间正在移动。

第三，忽视回归。Dat 在前文已经提醒，修复一个问题可能制造多个隐藏 regressions。生产治理必须把 regression guardrails 放在实验流程中，避免发布后才靠客服反馈发现。

第四，把自动化当成无条件放权。Alex 或 coding agent 可以协助分析和生成 eval，但自动行动需要权限控制和审计。自动化越强，治理机制越要具体。

最容易被遗漏的 unknown unknowns

团队常低估三类问题：一是 trace/span/session 的数据模型早期设计不良，后期很难补救；二是业务质量标准没有被领域专家明确写入 eval；三是自动化工具获得过多权限后，错误实验或错误修复会被快速放大。

7.4 一页落地清单

1. 明确最重要的生产任务：哪个 agent 或 harness 已经影响真实用户或业务流程。

2. 为关键路径建立 OpenTelemetry 风格的 traces、spans 和 sessions。
3. 选择最小有用 eval 集，覆盖格式、事实性、路径顺序、用户体验和业务结果。
4. 建立黄金数据集或可信标注样本，用于校准 LLM judge。
5. 把低质量 traces 收集成 datasets，保留 input-output pairs 和必要参考。
6. 对 prompts、models、orchestration 和 configurations 做受控实验。
7. 同时监控目标指标与 guardrail 指标，特别是延迟、成本、安全和业务指标。
8. 将 CLI/tools/skills 接入开发流程，让 coding agent 能运行评估和实验。
9. 对自动生成 eval 和自动修复设置权限、审计和人工复核。

7.5 最终综合

Dat 的演讲把一个容易被神秘化的问题讲得很朴素：AI 产品看起来像魔法，是因为模型输出具有开放性和不确定性；工程团队要做的是把开放性放回可观测、可评估、可实验的系统里。可观测性提供事实，评估提供判断，实验提供比较，自动化提供速度。

如果用一句中文压缩全篇，就是：生产 GenAI 的成熟度，不取决于 demo 中答案多惊艳，而取决于团队能否持续看见系统行为、解释质量信号、验证改动影响，并在治理边界内让自动化加速闭环。

7.6 本章小结

本章把整场演讲从产品演示抽象为生产治理模式。Dat 的核心信息是，observability、evaluation、experimentation and improvement 形成同一条工程链；Alex、CLI、tools 和 skills 的出现，是为了让这条链更自动、更快、更贴近开发流程。落地时，团队必须同时处理数据、信号、实验、自动化和治理，否则 “AI isn’t magic” 这句话只会停留在口号层面。